

- Description

This module manages an automated warehouse inventory system, ensuring that:

1. **Stock levels are continuously monitored** to prevent shortages.
2. **When stock falls below a predefined threshold, an automatic replenishment order is triggered.**
3. **Each replenishment order has a delivery time**, and stock is updated upon arrival.
4. **A record of all transactions (sales, replenishments) is maintained** for inventory tracking.

This system ensures efficient warehouse operations by maintaining optimal stock levels without manual intervention.

- Specification

module WarehouseInventoryManagement

var stock, quantity, reorderThreshold, reorderAmount: integer

var pendingOrders: sequence of integer

var transactionLog: sequence of string

process sell(quantity: integer) flag: boolean

functional scenario form:

quantity <= stock && flag = true and stock = stock – quantity and transactionLog = transactionLog $\oplus$ ("Sold: " + quantity + " items") and if stock < reorderThreshold then pendingOrders = pendingOrders $\oplus$ reorderAmount, Valid sale, stock updated, check for replenishment

||

quantity > stock && flag = false, Invalid sale, insufficient stock, no changes

end process

process receiveReplenishment()

functional scenario form:

(|pendingOrders| > 0) && stock = stock + first(pendingOrders) $\wedge$ transactionLog = transactionLog $\oplus$ "Replenishment received" $\wedge$ pendingOrders = removeFirst(pendingOrders), Replenishment received, stock updated.

OR

(|pendingOrders| = 0) && No pending replenishment orders.

end process

end module

- Java Code

```
package code;
import java.util.*;
class MyProduct {
    private String name;
    private int stock;
    private final int reorderThreshold;
```

```

    private final int reorderAmount;
    private Queue<ReplenishmentOrder> pendingOrders;
    private List<String> transactionLog;
    public MyProduct(String name, int initialStock, int reorderThreshold, int reorderAmount) {
        this.name = name;
        this.stock = initialStock;
        this.reorderThreshold = reorderThreshold;
        this.reorderAmount = reorderAmount;
        this.pendingOrders = new LinkedList<>();
        this.transactionLog = new ArrayList<>();
    }
    public void sell(int quantity) {
        if (quantity > 0 && quantity <= stock) {
            stock -= quantity;
            transactionLog.add(String.format("Sold: %d %s, Remaining stock: %d", quantity, name, stock));
            System.out.printf("Order fulfilled: Sold %d %s. Remaining stock: %d\n", quantity, name, stock);

            if (stock < reorderThreshold) {
                placeReplenishmentOrder();
            }
        } else {
            System.out.println("Order failed! Invalid quantity or insufficient stock.");
        }
    }
    private void placeReplenishmentOrder() {
        ReplenishmentOrder newOrder = new ReplenishmentOrder(reorderAmount);
        pendingOrders.add(newOrder);
        transactionLog.add(String.format("Replenishment order placed: + %d %s (ETA: %d hours)", reorderAmount, name, newOrder.getEta()));
        System.out.printf("Replenishment triggered! Ordered %d %s, ETA: %d hours\n", reorderAmount, name, newOrder.getEta());
    }
    public void receiveReplenishment() {
        if (!pendingOrders.isEmpty()) {
            ReplenishmentOrder order = pendingOrders.poll();
            stock += order.getQuantity();
            transactionLog.add(String.format("Replenishment received: + %d %s, New stock: %d", order.getQuantity(), name, stock));
        }
    }

```

```

        System.out.printf("Replenishment received! Added %d %s. New
stock: %d%n", order.getQuantity(), name, stock);
    } else {
        System.out.println("No pending replenishment orders.");
    }
}
// 显示库存交易日志
public void printTransactionLog() {
    System.out.println("\nTransaction History for " + name + ":");
    for (String log : transactionLog) {
        System.out.println(log);
    }
}
}
class ReplenishmentOrder {
    private final int quantity;
    private final int eta;
    public ReplenishmentOrder(int quantity) {
        this.quantity = quantity;
        this.eta = new Random().nextInt(24) + 1;
    }
    public int getQuantity() {
        return quantity;
    }
    public int getEta() {
        return eta;
    }
}
public class c13_WarehouseInventorySystem {
    public static void main(String[] args) {
        MyProduct laptop = new MyProduct("Laptop", 10, 5, 20);

        // testcase-VT:
        //     laptop.sell(3);
        //     laptop.sell(4);
        //     laptop.sell(2);
        //     laptop.sell(-2);
        //     laptop.receiveReplenishment();
        //     laptop.sell(5);
        //     laptop.sell(100);
        //     laptop.sell(234);
        //     laptop.sell(63);
        //     laptop.sell(302);
        //     laptop.sell(459);
    }
}

```

```
//      laptop.sell(108);
//      laptop.sell(254);
//      laptop.sell(331);
//      laptop.sell(79);
//      laptop.sell(316);
//      laptop.sell(63);
//      laptop.sell(220);
//      laptop.sell(47);
//      laptop.sell(361);
//      laptop.sell(189);
//      laptop.sell(263);
//      laptop.sell(59);
//      laptop.sell(33);
//      laptop.sell(485);
//      laptop.sell(373);
//      laptop.sell(271);
//      laptop.sell(125);
//      laptop.sell(172);
//      laptop.sell(82);
//      laptop.sell(331);
//      laptop.sell(178);
//      laptop.sell(448);
//      laptop.sell(496);
//      laptop.sell(431);
//      laptop.sell(304);
//      laptop.sell(455);
//      laptop.sell(121);
//      laptop.sell(463);
//      laptop.sell(292);
//      laptop.sell(389);
//      laptop.sell(248);
//      laptop.sell(150);
//      laptop.sell(36);
//      laptop.sell(118);
//      laptop.sell(173);
//      laptop.sell(29);
//      laptop.sell(119);
//      laptop.sell(267);
//      laptop.sell(364);
//      laptop.sell(130);
//      laptop.sell(167);
//      laptop.sell(270);
//      laptop.sell(294);
//      laptop.sell(103);
```

```
//      laptop.sell(59);
//      laptop.sell(401);
//      laptop.sell(295);
//      laptop.sell(240);
//      laptop.sell(282);
//      laptop.sell(304);
//      laptop.sell(72);
//      laptop.sell(236);
//      laptop.sell(218);
//      laptop.sell(284);
//      laptop.sell(359);
//      laptop.sell(84);
//      laptop.sell(129);
//      laptop.sell(137);
//      laptop.sell(239);
//      laptop.sell(150);
//      laptop.sell(218);
//      laptop.sell(478);
//      laptop.sell(92);
//      laptop.sell(307);
//      laptop.sell(40);
//      laptop.sell(55);
//      laptop.sell(36);
//      laptop.sell(259);
//      laptop.sell(482);
//      laptop.sell(486);
//      laptop.sell(256);
//      laptop.sell(94);
//      laptop.sell(251);
//      laptop.sell(29);
//      laptop.sell(187);
//      laptop.sell(413);
//      laptop.sell(308);
//      laptop.sell(364);
//      laptop.sell(313);
//      laptop.sell(500);
//      laptop.sell(90);
//      laptop.sell(21);
//      laptop.sell(50);
//      laptop.sell(310);
//      laptop.sell(186);
//      laptop.sell(276);
//      laptop.sell(471);
//      laptop.sell(361);
```

```
//      laptop.sell(211);
//      laptop.sell(244);
//      laptop.receiveReplenishment();

      //testcase-FT:
//      laptop.sell(1);
//      laptop.sell(16776961);
//      laptop.sell(-65219);
//      laptop.sell(1040122173);
//      laptop.sell(64);
//      laptop.sell(64);
//      laptop.sell(64);
//      laptop.sell(16448);
//      laptop.sell(64);
//      laptop.sell(1128481603);
//      laptop.sell(1128481603);
//      laptop.sell(64);
//      laptop.sell(70);
//      laptop.sell(1186182067);
//      laptop.sell(192);
//      laptop.sell(192);
//      laptop.sell(0);
//      laptop.sell(0);
//      laptop.sell(0);
//      laptop.sell(58);
//      laptop.sell(989855743);
//      laptop.sell(989855743);
//      laptop.sell(64);
//      laptop.sell(51);
//      laptop.sell(65);
//      laptop.sell(65);
//      laptop.sell(67);
//      laptop.sell(64);
//      laptop.sell(72);
//      laptop.sell(18496);
//      laptop.sell(18478);
//      laptop.sell(18505);
//      laptop.sell(15680);
//      laptop.sell(15707);
//      laptop.sell(23357);
//      laptop.sell(4217661);
//      laptop.sell(4201277);
//      laptop.sell(64);
//      laptop.sell(64);
```

```
//      laptop.sell(86);
//      laptop.sell(96);
//      laptop.sell(64);
//      laptop.sell(-1);
//      laptop.sell(-1);
//      laptop.sell(-1);
//      laptop.sell(-1);
//      laptop.sell(3648);
//      laptop.sell(3648);
//      laptop.sell(3584);
//      laptop.sell(250276586);
//      laptop.sell(250276586);
//      laptop.sell(66);
//      laptop.sell(1119796926);
//      laptop.sell(1119796926);
//      laptop.sell(1119796926);
//      laptop.sell(1119796775);
//      laptop.sell(16448);
//      laptop.sell(4210752);
//      laptop.sell(-1405075392);
//      laptop.sell(-1405047744);
//      laptop.sell(-1400996226);
//      laptop.sell(0);
//      laptop.sell(0);
//      laptop.sell(16384);
//      laptop.sell(1077936128);
//      laptop.sell(1077936326);
//      laptop.sell(64);
//      laptop.sell(16448);
//      laptop.sell(16576);
//      laptop.sell(49344);
//      laptop.sell(-1061141436);
//      laptop.sell(16448);
//      laptop.sell(16500);
//      laptop.sell(29812);
//      laptop.sell(3896436);
//      laptop.sell(29812);
//      laptop.sell(65);
//      laptop.sell(59);
//      laptop.sell(16443);
//      laptop.sell(1077673983);
//      laptop.sell(16448);
//      laptop.sell(20544);
//      laptop.sell(20544);
```

```
//      laptop.sell(5259327);
//      laptop.sell(1065023);
//      laptop.sell(1073741824);
//      laptop.sell(1073741824);
//      laptop.sell(1086850561);
//      laptop.sell(16609);
//      laptop.sell(16865);
//      laptop.sell(2);
//      laptop.sell(205);
//      laptop.sell(205);
//      laptop.sell(48);
//      laptop.sell(50);
//      laptop.sell(13049);
//      laptop.sell(13305);
//      laptop.sell(3406116);
//      laptop.sell(16448);
//      laptop.sell(64);

      // testcase-Z3:
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.receiveReplenishment();
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
```

[illegible]

```
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.sell(5);
//      laptop.sell(0);
//      laptop.sell(3);
//      laptop.sell(4);
//      laptop.sell(2);
//      laptop.sell(2);
//      laptop.sell(1);
//      laptop.receiveReplenishment();
```

```
// testcase-UVT:
```

```
laptop.sell(3);
laptop.sell(4);
laptop.sell(2);
laptop.sell(2);
laptop.receiveReplenishment();
laptop.sell(5);
laptop.sell(100);
laptop.sell(234);
laptop.sell(63);
```

```
laptop.sell(302);
laptop.sell(459);
laptop.sell(108);
laptop.sell(254);
laptop.sell(331);
laptop.sell(79);
laptop.sell(316);
laptop.sell(63);
laptop.sell(220);
laptop.sell(47);
laptop.sell(361);
laptop.sell(189);
laptop.sell(263);
laptop.sell(59);
laptop.sell(33);
laptop.sell(485);
laptop.sell(373);
laptop.sell(271);
laptop.sell(125);
laptop.sell(172);
laptop.sell(82);
laptop.sell(331);
laptop.sell(178);
laptop.sell(448);
laptop.sell(496);
laptop.sell(431);
laptop.sell(304);
laptop.sell(455);
laptop.sell(121);
laptop.sell(463);
laptop.sell(292);
laptop.sell(389);
laptop.sell(248);
laptop.sell(150);
laptop.sell(36);
laptop.sell(118);
laptop.sell(173);
laptop.sell(29);
laptop.sell(119);
laptop.sell(267);
laptop.sell(364);
laptop.sell(130);
laptop.sell(167);
laptop.sell(270);
```

```
laptop.sell(294);  
laptop.sell(103);  
laptop.sell(59);  
laptop.sell(401);  
laptop.sell(295);  
laptop.sell(240);  
laptop.sell(282);  
laptop.sell(304);  
laptop.sell(72);  
laptop.sell(236);  
laptop.sell(218);  
laptop.sell(284);  
laptop.sell(359);  
laptop.sell(84);  
laptop.sell(129);  
laptop.sell(137);  
laptop.sell(239);  
laptop.sell(150);  
laptop.sell(218);  
laptop.sell(478);  
laptop.sell(92);  
laptop.sell(307);  
laptop.sell(40);  
laptop.sell(55);  
laptop.sell(36);  
laptop.sell(259);  
laptop.sell(482);  
laptop.sell(486);  
laptop.sell(256);  
laptop.sell(94);  
laptop.sell(251);  
laptop.sell(29);  
laptop.sell(187);  
laptop.sell(413);  
laptop.sell(308);  
laptop.sell(364);  
laptop.sell(313);  
laptop.sell(500);  
laptop.sell(90);  
laptop.sell(21);  
laptop.sell(50);  
laptop.sell(310);  
laptop.sell(186);  
laptop.sell(276);
```

```

        laptop.sell(471);
        laptop.sell(361);
        laptop.sell(211);
        laptop.sell(244);
        laptop.receiveReplenishment();
        laptop.printTransactionLog();

    }
}

```

- Coverage report

1. Test case number: 100

2. V-Method testing branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.1 %	9	825	834
▼ src	1.1 %	9	825	834
▼ code	1.1 %	9	825	834
▼ c13_WarehouseInventorySystem.java	90.0 %	9	1	10
> MyProduct	90.0 %	9	1	10
> c13_WarehouseInventorySystem		0	0	0
> ReplenishmentOrder		0	0	0

3. Vibration testing instruction coverage: 99%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	3.6 %	578	15,470	16,048
▼ src	3.6 %	578	15,470	16,048
▼ code	3.6 %	578	15,470	16,048
▼ c13_WarehouseInventorySystem.java	99.0 %	578	6	584
> c13_WarehouseInventorySystem	99.1 %	315	3	318
> MyProduct	98.8 %	242	3	245
> ReplenishmentOrder	100.0 %	21	0	21

4. Fuzz testing branch coverage: 70%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.8 %	7	827	834
▼ src	0.8 %	7	827	834
▼ code	0.8 %	7	827	834
▼ c13_WarehouseInventorySystem.java	70.0 %	7	3	10
> MyProduct	70.0 %	7	3	10
> c13_WarehouseInventorySystem		0	0	0

5. Fuzz testing instruction coverage: 73.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	2.6 %	424	15,620	16,044
▼ src	2.6 %	424	15,620	16,044
▼ code	2.6 %	424	15,620	16,044
▼ c13_WarehouseInventorySystem.java	73.1 %	424	156	580
> MyProduct	46.1 %	113	132	245
> c13_WarehouseInventorySystem	99.0 %	311	3	314

6. Z3 branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.1 %	9	831	840
▼ src	1.1 %	9	831	840
▼ code	1.1 %	9	831	840
▼ c13_WarehouseInventorySystem.java	90.0 %	9	1	10
> MyProduct	90.0 %	9	1	10
> c13_WarehouseInventorySystem		0	0	0
> ReplenishmentOrder		0	0	0

7. Z3 instruction coverage: 99%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	2.6 %	578	21,515	22,093
▼ src	2.6 %	578	21,515	22,093
▼ code	2.6 %	578	21,515	22,093
▼ c13_WarehouseInventorySystem.java	99.0 %	578	6	584
> MyProduct	98.8 %	242	3	245
> c13_WarehouseInventorySystem	99.1 %	315	3	318
> ReplenishmentOrder	100.0 %	21	0	21

8. Unimproved V-method branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	832	840
▼ src	1.0 %	8	832	840
▼ code	1.0 %	8	832	840
▼ c13_WarehouseInventorySystem.java	80.0 %	8	2	10
> MyProduct	80.0 %	8	2	10
> c13_WarehouseInventorySystem		0	0	0
> ReplenishmentOrder		0	0	0

9. Unimproved V-method instruction coverage: 99%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	2.6 %	578	21,515	22,093
▼ src	2.6 %	578	21,515	22,093
▼ code	2.6 %	578	21,515	22,093
▼ c13_WarehouseInventorySystem.java	99.0 %	578	6	584
> MyProduct	98.8 %	242	3	245
> c13_WarehouseInventorySystem	99.1 %	315	3	318
> ReplenishmentOrder	100.0 %	21	0	21